

Cheatsheet——考试加油！

一、语法与常识

1、集合 set

```
#创建集合
my_set = {1, 2, 3, 4, 5}
another_set = set([3, 4, 5, 6, 7])
#注意: set() 用来创建集合时, 它接受一个可迭代对象(如列表、元组、字符串等), 因而这里set() 会自动从列表中提取元素并创建集合, 而不能直接set(3, 4, 5, 6, 7), 因为set()括号里只可以有一个参数, 而{}则不同。
# 添加元素
my_set.add(6)
# 删除元素(不存在元素可抛出错误)
my_set.remove(2)
# 删除不存在的元素, 不会抛出错误
my_set.discard(10)
```

2、ASCII码

```
ord(): char -> int
chr(): int -> char
#对应情况有:
10 ->NL, line feed, new line
13 ->CR, carriage return
48 ->'0'; 49 ->'1'
65 ->'A'; 97 ->'a'
```

3、为枚举对象编号 enumerate

无需import直接使用

```
enumerate(interable, start:int =0)
```

4、堆 heapq (本质是二叉树)

```
import heapq
#创建与推入
heapq.heapify(list)->None #将原先的列表建立为堆
heapq.merge(*iterables)->iterable #将有序序列合并排序, 返回可迭代
heapq.heappush(heap, item)
#访问与弹出
heapq.nlargest(n:int, interable[, key])->list #访问前n大元素
heapq.nsmallest(n, interable[, key])->list #访问前n小元素
heapq.heappop(heap)->item #弹出并返回最小元素
heapq.heapreplace(heap, item)->None #删除最小元素并加入一个元素
heapq.heappushpop(heap, item)->item #同时push+pop
#堆排(不如sorted, 略)
#可用于反悔写法
```

5、两端队列 deque

```
from collections import deque
q=deque()
q.append(1)
q.appendleft(2)
x= q.pop()
y= q.popleft()
q.extend([4,5])
q.extendleft([5,4]) #注意左端添加元素时是逆序添加
```

6、二分查找 bisect

```
import bisect
bisect.bisect_left(interable, item)->int
#返回一个索引，该索引是列表 arr 中插入元素 x 的位置，并且会确保 x 插入后，列表仍然保持升序排列。如果 x 已经存在于列表中，则返回左边的插入位置（即 x 的第一个位置）。
bisect.insort_left(interable, item)->None
#将元素 x 插入到列表 arr 中，并保持列表的升序。若x已经存在，插入左边。
```

7、深拷贝

```
import copy
deep_copy_list = copy.deepcopy(original)
```

8、迭代器 itertools

```
import itertools
itertools.permutations(iterable,n:int)->list[tuple] #n元排列
itertools.combinations(iterable,n:int)->list[tuple] #n元组合
```

二、算法模板

1、欧拉筛

```
def oula(r):
    is_prime = [0 for i in range(r+1)]
    common = []
    for i in range(2, r+1):
        if is_prime[i] == 0:
            common.append(i)
            for j in common:
                if i*j > r:
                    break
                is_prime[i*j] = 1
                if i % j == 0:
                    break
    return common
```

2、背包问题

```

#01背包
n,b=map(int, input().split())
price=[int(i) for i in input().split()] # (注意这里下标从零开始)
weight=[int(i) for i in input().split()]
bag=[[0]*(b+1) for _ in range(n+1)]
for i in range(1,n+1):
    for j in range(1,b+1):
        if weight[i]<=j:
            bag[i][j]=max(price[i]+bag[i-1][j-weight[i]], bag[i-1][j])
        else:
            bag[i][j]=bag[i-1][j]
print(bag[-1][-1])

#完全背包 (无数个)
n, a, b, c = map(int, input().split())
dp = [float('-inf')]*n
for i in range(1, n+1):
    for j in (a, b, c):
        if i >= j:
            dp[i] = max(dp[i-j] + 1, dp[i])
print(dp[n])

#多重背包二进制优化
将s[i]件物品分成log2(s[i])个袋子，分别装1, 2, 4, 8.....个，将多重背包转化为01背包

```

3、bfs模板

```

from collections import deque
def bfs(start_x,start_y):
    q=deque([0,start_x,start_y])
    in_queue={(start_x,start_y)}

    while q:
        step,x,y=q.popleft()
        if 达到终点:
            return step
        for 坐标(i,j) in 可能的坐标:
            if 坐标合理且不在in_queue中:
                in_queue.add((i,j))
                q.append((step+1,i,j))
    return 错误反馈

```

4、dfs模板

```

def dfs(原始集合, 递归层数, 状态变量, 结果集合):
    #标记当前位置
    if 终止条件:
        #打印或添加到结果集
        return
    for 所有可能的分支路径:
        if 剪枝条件(层数, 状态变量的条件etc):
            continue
        #操作状态变量
        dfs()
        #回溯状态变量

    return

```

5、dijkstra

最短权值路径（现有一个共n个顶点（代表城市）、m条边（代表道路）的无向图（假设顶点编号为从0到n-1），每条边有各自的边权，代表两个城市之间的距离。求从s号城市出发到达t号城市的最短距离。

```

def dijkstra(n, edges, s, t):
    graph = [[] for _ in range(n)]
    for u, v, w in edges:
        graph[u].append((v, w))
        graph[v].append((u, w))
    pq = [(0, s)] # (distance, node)
    visited = set()
    distances = [float('inf')] * n
    distances[s] = 0
    while pq:
        dist, node = heapq.heappop(pq)
        if node == t:
            return dist
        if node in visited:
            continue
        visited.add(node)
        for neighbor, weight in graph[node]:
            if neighbor not in visited:
                new_dist = dist + weight
                if new_dist < distances[neighbor]:
                    distances[neighbor] = new_dist
                    heapq.heappush(pq, (new_dist, neighbor))
    return -1
n, m, s, t = map(int, input().split())
edges = [list(map(int, input().split())) for _ in range(m)]
result = dijkstra(n, edges, s, t)
print(result)

```